



# Práctica Extra 4

Organización y Arquitectura de Computadoras

Docente: M.C. Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

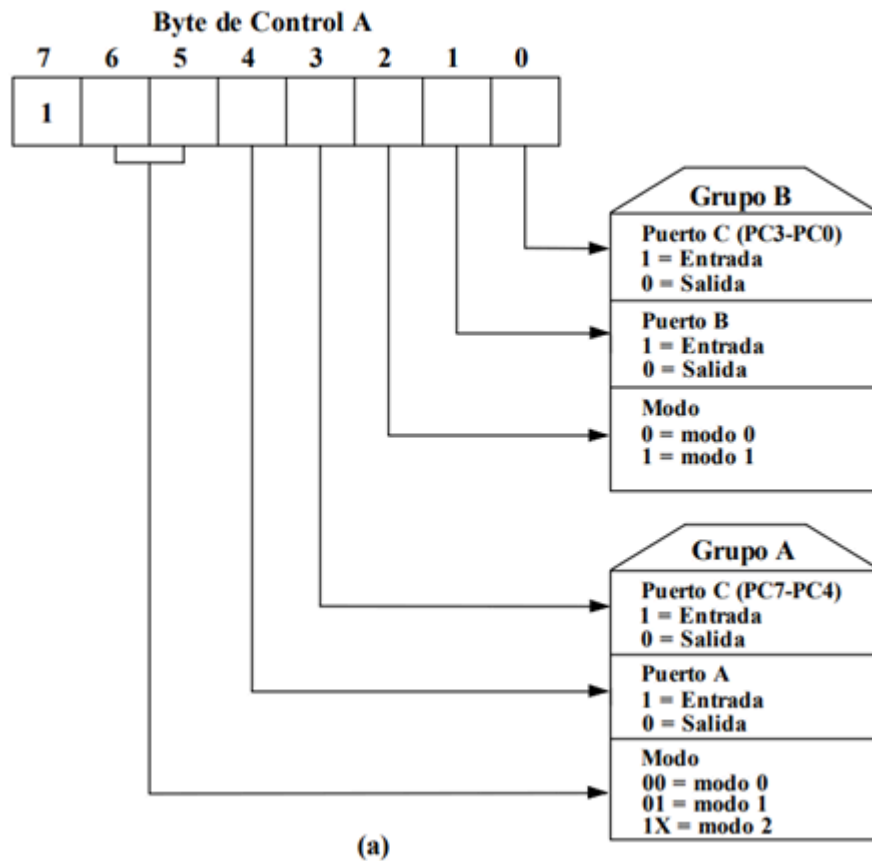
Tijuana, Baja California a 30 de noviembre de 2025



## Objetivo

Acceso a los puertos genéricos del dispositivo programable PPI-8255 que se encuentra emulado en la tarjeta T-Juino, además, el alumno se familiarizará con el intérprete 80X86 (modo 16bits) sobre la plataforma T-Juino. Esto con el fin de desarrollar programas en lenguaje C para dicha plataforma.

## Desarrollo



Tomando en cuenta la figura a) configurar el byte de control para tener, agregar las líneas de código de dicha configuración en asm:

- El puerto C parte baja de entrada y la parte alta como entrada
- El puerto B como salida.
- El puerto A como salida.

Palabra de control: 1000 1001 (89h)

Actividad 1:

1. Desarrollar una subrutina llamada `inportb` en `asm` para ingresar datos de 8 bits por un puerto seleccionado por `DX` y este dato retornarlo en `AL`.

```
33 inportb proc ; DX recibe el puerto | Retorna el dato en AL
34     in al, dx
35     ret
36 endp
```

- a. Para probar esta parte se requiere ingresar 1 número de un byte en el puerto seleccionado y sacar ese valor leído de manera invertida con `outportb`, ejemplo entrada 01010100 y sale 10101011

1. Crear un archivo llamado `ejemploC.c` con el siguiente código:

```
2  #define true 1
3  #define false !true
4
5  typedef unsigned int word;
6  typedef unsigned char byte;
7
8  extern int suma(word A, word B, word C);
9  void printdec (word dato);
10 extern void putchar (byte dato);
11 byte getchar (void);
12 void puts(byte *);
13 byte capturarNum(void);
14
15 byte *msg1={"Ingrese el primer valor A del 0-9:"};
16 byte *msg2={"Ingrese el segundo valor C del 0-9:"};
17 byte *msg3={"Ingrese el modo de suma B del 0-> A+C y 1-> A+C+1:"};
18 byte *msg4={"El resultado es:"};
19 byte *msg5={"Presionar cualquier tecla para repetir.\r\n"};
20 byte *msg6={"\r\n"};
21
22
23 void main( void ){
24     word A, B, C, Res;
25
26
27     while(true){
28         puts(msg1);
29         A = capturarNum();
30         puts(msg6);
31         puts(msg2);
32         C = capturarNum();
33         puts(msg6);
34         puts(msg3);
35         B = capturarNum();
36         puts(msg6);
37         Res = suma(A,B,C);
38         puts(msg4);
39         printdec(Res);
40         puts(msg6);
41         puts(msg5);
42         getchar();
43     }
44 }
45
46 void printdec (word dato){
47     putchar(dato/100+0x30);
48     dato%=100;
49     putchar(dato/10+0x30);
50     putchar(dato%10+0x30);
51 }
52
53 /**void putchar(byte dato){
54     byte tmp;
55     do{
56         tmp = getchar();
57     }while(tmp < '0' || tmp > '9');
58     return tmp-48;
59 }
60
61 byte getchar (void){
62     byte dato;
63     asm mov ah, 1
64     asm int 0x21
65     asm mov dato, al
66     return dato;
67 }
68
69 void puts(byte *str){
70     while(*str)
71         putchar(*str++);
72 }
73
74 void puts(byte *str){
75     while(*str)
76         putchar(*str++);
77 }
```



2. Crear un archivo llamado eASM.asm e introducir el siguiente código:

```
1  .model tiny
2  .code
3
4      public _putchar
5      public _suma
6
7  _putchar proc
8      push bp
9      mov bp, sp
10     push dx
11     push ax
12     mov dl, [bp+4]
13     mov ah, 2
14     int 21h
15     pop ax
16     pop dx
17     pop bp
18     ret
19 _putchar endp
20
21 ;Representacion de pila despues del mov bp, sp de proc suma
22 ;SP->|Valor de BP |BP
23 ;SP+2|Direccion |BP+2
24 ;    |de retorno |
25 ;SP+4|Valor de A |BP+4
26 ;SP+6|Valor de B |BP+6
27 ;SP+8|Valor de C |BP+8
28 ;SP+10|Desconocido |BP+10
29
30 _suma PROC
31     push bp
32     mov bp, sp
33
34     mov ax, [bp+4] ; parametro A en AX de suma(A, B, C)
35     mov bx, [bp+6] ; parametro B en BX de suma(A, B, C)
36     mov cx, [bp+8] ; parametro C en CX de suma(A, B, C)
37
38     ;if BX == 0 then suma A+C
39     ;if BX == 1 then suma A+C+B
40
41     cmp bl, 0
42     je .solo_suma
43     add ax, bx
44
45 .solo_suma: add ax, cx ;esta aqui ya que es una
46             ;instruccion comun entre ambos caminos
47
48     ;retornamos el resultado en ax
49     ;osea es de 16 bits, de 16 bits
50     ;tenemos int en c
51     pop bp
52     ret
53     ENDP
54 END
```



3. Crear una carpeta en la raíz llamada oac y ahí dejar los archivos anteriores, aparte se ocupará ahí mismo los archivos initj.obj, tcc, tasm, mkbintj.exe, tlink (los cuales deben bajar de moodle).

- 1) Para proceder se requiere instalar DosBox:
  - a) Crear una carpeta en el disco C: (el raíz) llamada OAC.
  - b) Instalar DosBox (sigan algún tutorial).
  - c) Al iniciar DosBox ejecutar lo siguiente:

```
mount c: c:/oac  
c:
```

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX  
Welcome to DOSBox v0.74-3  
For a short introduction for new users type: INTRO  
For supported shell commands type: HELP  
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.  
To activate the keymapper ctrl-F1.  
For more information read the README file in the DOSBox directory.  
HAVE FUN!  
The DOSBox Team http://www.dosbox.com  
Z:\>SET BLASTER=A220 I7 D1 H5 T6  
Z:\>mount c: c:/oac  
Drive C is mounted as local directory c:/oac\  
Z:\>c:  
C:\>
```

- 2) Descargue los programas tcc.exe, tasm.exe, tlink.exe y mkbintj.exe de moodle y deposítelos en undirectorio exclusivo de trabajo (por ejemplo. C:\OAC\P13).

| Name        | Date modified      | Type        | Size   |
|-------------|--------------------|-------------|--------|
| INITJ.OBJ   | 8/26/2016 4:06 PM  | Object File | 1 KB   |
| mkbintj.exe | 8/26/2016 4:06 PM  | Application | 18 KB  |
| TASM.EXE    | 3/20/2010 12:02 AM | Application | 104 KB |
| TCC.EXE     | 3/20/2010 12:02 AM | Application | 177 KB |
| TLINK.EXE   | 5/11/1989 1:01 AM  | Application | 22 KB  |



- 3) Compile el programa ejemploC.c mediante la línea de comando: C:\OAC\P13>tcc-c ejemploC.c Esto generará el archivo ejemploC.obj

```
C:\PEXTRA4>tcc -c ejemploC.c
Turbo C Version 2.01 Copyright (c) 1987, 1988 Borland International
ejemploC.c:
Warning ejemploC.c 64: Restarting compile using assembly in function getchar
Turbo Assembler Version 2.01 Copyright (c) 1988, 1990 Borland International

Assembling file:  ejemploC.ASM
Error messages:  None
Warning messages: None
Passes:         1
Remaining memory: 303k

                Available memory 445106

C:\PEXTRA4>
```

| Name           | Date modified       | Type        | Size   |
|----------------|---------------------|-------------|--------|
| INIT.OBJ       | 8/26/2016 4:06 PM   | Object File | 1 KB   |
| mkbintj.exe    | 8/26/2016 4:06 PM   | Application | 18 KB  |
| TASM.EXE       | 3/20/2010 12:02 AM  | Application | 104 KB |
| TCC.EXE        | 3/20/2010 12:02 AM  | Application | 177 KB |
| TLINK.EXE      | 5/11/1989 1:01 AM   | Application | 22 KB  |
| EjemploASM.asm | 11/30/2025 11:03 PM | ASM File    | 1 KB   |
| EjemploC.c     | 11/30/2025 10:56 PM | C File      | 2 KB   |
| EJEMPLOC.OBJ   | 12/1/2025 12:43 AM  | Object File | 1 KB   |

- 4) Ensamble la librería eASM.asm mediante la línea de comando: C:\OAC\P13>tasm-c eASM.asm Esto generará el archivo eASM.obj

```
C:\PEXTRA4>tasm -c EASM.ASM
Turbo Assembler Version 2.01 Copyright (c) 1988, 1990 Borland International

Assembling file:  EASM.ASM
Error messages:  None
Warning messages: None
Passes:         1
Remaining memory: 492k
```



- 5) Encadena los archivos obj para generar programa .exe mediante la línea de comando:  
C:\OAC\P13>tlink initj.obj ejemploC.obj eASM.obj, ejemplo.exe Esto generará el  
archivo ejecutable ejemplo.exe

```
C:\PEXTRA4>tlink initj.obj EJEMPLOC.OBJ EASM.obj, ejemplo.exe
Turbo Link Version 2.0 Copyright (c) 1987, 1988 Borland International
Warning: no stack
```

- 6) Convertir archivo ejemplo.exe a formato binario para la placa T-Juino haciendo uso de la herramienta mkbintj.exe

```
C:\OAC\P13>mkbintj.exe ejemplo.exe
```

Esto generará el archivo ejemplo.bin el cual puede cargarse y ejecutarse en T-Juino.

```
C:\PEXTRA4>mkbintj.exe ejemplo.exe
Archivo EJEMPLO.BIN para SM8088 ha sido creado
```

- 7) Verifique el funcionamiento del programa cargándolo y ejecutándolo en T-Juino.  
Esto genera un archivo .bin el cual se debe cargar al Tjuino y con ello ya estarán uniendo C con ASMdel16bits, con todas las ventajas de poder acceder a los puertos físicos del dispositivo.

```
>g
Ingrese el primer valor A del 0-9:5
Ingrese el segundo valor C del 0-9:6
Ingrese el modo de suma B del 0 -> (A + C) y 1 -> (A + C + 1):0
El resultado es:011
Presionar cualquier tecla para repetir.
```



## **Código en ASM**





## Dificultades durante el desarrollo

Durante el desarrollo de la práctica,

## Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)